从提需求到部署发布，全AI全自动化后，研发效能全面跃升

原创 腾讯程序员 腾讯技术工程 2026年4月20日 02:36 广东 428人



作者：jacksondeng、yuxuanlai、gcchaoguo

1、背景与挑战

1.1 背景

当前从需求到交付的全链路中，大量人工仍耗费在方案设计、代码评审、测试验证等非编码环节，频繁的上下文切换与协作等待严重制约了研发效能。为此，我们在深耕AI编码能力的同时，正将自动化向上游的需求、方案设计与下游的测试、部署环节延伸，旨在通过AI将各环节无缝串联，最终实现需求至部署的全自动化交付与研发效能的全面跃升。

1.2现状

审核将AI全自动化演进规划为L1（纯人工）、L2（人机协同）、L3（全自动）三个阶段。

2025年，我们处于以“技术方案生成代码”为核心的L2阶段。基于“研发效能提升需突破单点、覆盖全链路”的共识，我们正从聚焦Coding，向需求、测试、部署等环节系统化辐射。最终目标是借由AI将各环节无缝串联，构建需求至部署的端到端自动化交付能力，驱动整体研发效能实现80%的效能提升，具体演进路线可以参考图1.1所示：

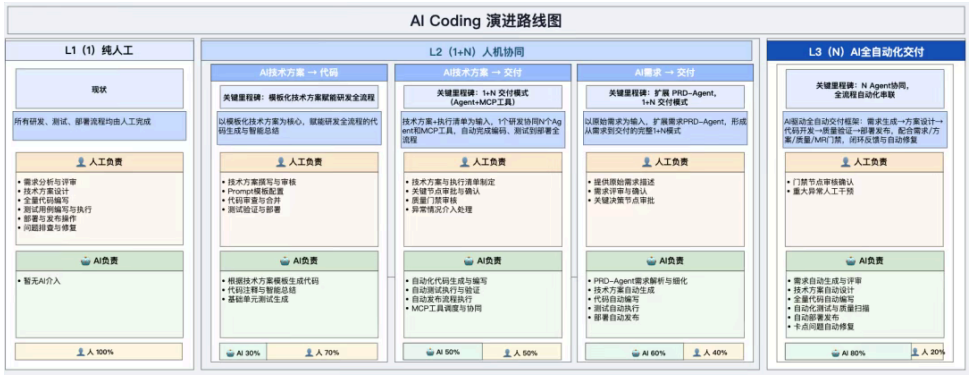


图1.1 演进路线图

审核迈向L3需经历三段式能力阶梯：先夯实技术方案生成代码，再打通技术方案至交付链路，最终实现需求到交付的端到端跑通。在此基础之上，我们才会全面进入L3的全自动化交付阶段。详细信息如下：

阶段	关键里程碑	简要描述	效能分布（AI：人工）
L1（1） 纯人工	研发人工完成编码	所有研发、测试、部署流程均由人工主导完成	0%：100%
L2（1+N） 人机协同	AI技术方案到代码	以模板化的技术方案为核心，赋能研发全流程的代码生成与智能总结	30%：70%
	AI技术方案到交付	以技术方案 + 执行清单（测试执行 + 发布执行清单）为输入，由1个研发协同N个Agent和MCP工具，自动完成编码、测试到部署的全流程，形成 “1+N”交付模式	50%：50%
	AI需求到交付	以原始需求为输入，在原来的基础上扩展了需求PRD-Agent，形成 “1+N”交付模式	70%：30%
L3（N） AI全自动化交付	静态功能全自动化交付	构建AI驱动的全自动化交付框架，实现从需求生成、方案设计、代码开发、质量验证到部署发布的全流程自动化串联，并通过设置需求评审、方案评审、质量门禁与MR等门禁，构建闭环反馈与卡点自动修复能力，保障流程的稳定自适应和交付质量，形成“N Agent协同交付”模式	80% ： 20%
	运行时架构/性能/可用性问题自动化交付	构建与交付框架并行的AI全自动化治理框架，打通运行时日志和监测指标，实现线上服务的自动巡检与修复、线上异常的实时感知与优化，从而完成从架构、性能到可用性的系统性治理。最终形成“交付驱动效率，治理保障质量”的双轮驱动体系，确保AI研发全流程的可持续演进	

1.3 挑战

审核要全面演进至L3阶段，需系统应对四大核心挑战：一是交付流程的标准化建设，二是需求描述的规范化与结构化，三是高质量、可检索的知识库搭建，四是核心技能（Skills）的标准化与复用。详细信息如下：

挑战	关键举措
交付流程的标准化	制定一套标准的研发流程，让大模型按照标准流程执行，同时具备流程回滚能力 ● 短期：通过流程编排skill，为大模型制定标准的交付流程，将需求、研发、测试、部署流程全自动串联起来，并且支持出现问题，流程自动回滚的能力 ● 长期：通过流程编排管理页面，配置交付流程及流程对应的相关skills
需求的标准化	让需求按照标准模板输出，大模型可以基于需求生成代码 ● 制定标准的需求模板，主要通过需求概述、需求正文、版本管理、外部依赖等4个部分对需求文档进行规范性约束，提升需求的质量 ● 制定需求的澄清规则，通过对需求概述、需求正文、版本管理、外部依赖等4个部分，建立一套需求澄清的规则，在需求编写不满足条件的情况下，给出修正建议 ● 制定需求评分规则，通过对需求背景、价值衡量、解决方案、影响范围、需求正文设定一套标准的打分规则，方便衡量需求的输出质量，如果不满足评分标准，需

挑战	关键举措
	求将被打回 ● 构建需求PRD-Agent，配置需求模板、需求澄清规则、需求评分规则，支持产品在PRD-Agent上进行需求编写、澄清等
skills技能的沉淀/标准化	沉淀一套标准的skill能力，增强模型的执行和决策能力，目前需要沉淀技术方案生成、代码生成、代码CR、测试用例执行等skill能力
知识库的搭建	从需求生成代码，需要高质量的上下文 ● 构建审核的业务知识库，让大模型可以基于知识库生成技术方案 ● 构建审核的代码知识库，让大模型可以快速完成代码的熟悉，从而生成高质量的代码
代码质量保障	AI编码在提升速度的同时，也带来了质量失控的风险。若缺乏严格的质量门禁，代码库将在高速迭代中滑向高熵与腐化 ● 构建需求评审、方案评审、代码CR、安全检测等门禁，确保代码质量
线上问题的治理	AI编码基本都在静态代码的生成，对于系统架构、性能、可用性等问题会稍显不足，需要我们有一套对线上系统的实时监控及自我修复能力 ● 构建全自动化治理框架，打通运行时日志和监测指标，实现线上服务的自动巡检与修复、线上异常的实时感知与优化，从而完成从架构、性能到可用性的系统性治理

2、详细实践

我们致力于打造一个能够实现需求至发布全链路自动化、并具备自愈能力的AI交付平台。其成熟标志是形成由多个AI智能体自主协同的“N Agent交付模式”，达成L3级全自动化。为此，我们确立了分阶段演进的路线：现阶段（L2）的核心是“人机协同”，即全力打通并集成研发、测试、部署等关键工具链（MCP），为高阶自动化奠定工程基础。待此阶段稳固后，再系统性地迈向L3的AI全自动化交付阶段。

2.1.人机协同（L2）

2.1.1 技术方案到代码

在过去一年我们聚焦在coding阶段，主要通过“AI技术规范(索引/rules) + 模板化(技术方案/prompt) + MCP工具集成 + AI自我总结”，实现生产力提升的人机协同新范式。详情可以参考《AI研发新范式：基于技术方案全链路生成代码》，具体流程如下：

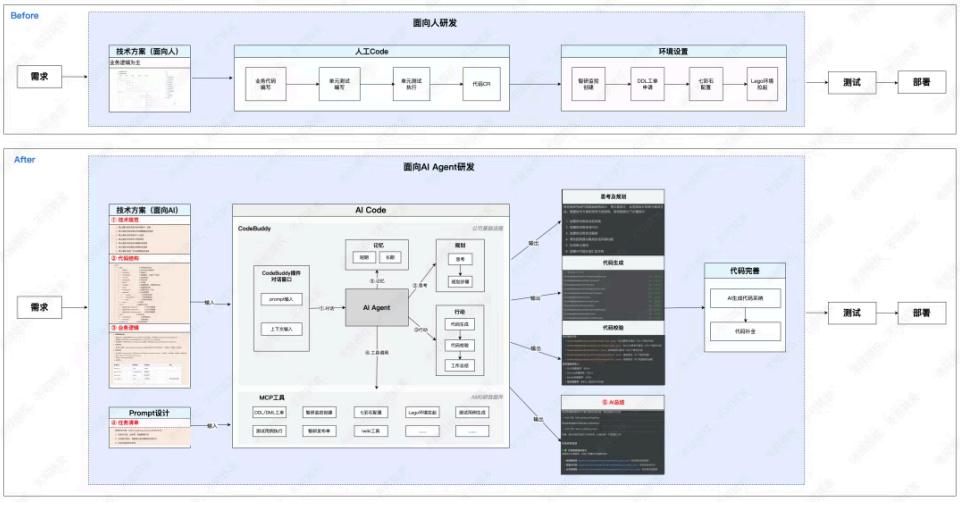


图2.1 审核面向AI Agent研发流程图

但是从需求到交付的全链路中，大量人工仍耗费在方案设计、代码评审、测试验证等非编码环节，所以我们将目光逐步向上游的需求、方案设计与下游的测试、部署环节延伸。

2.1.2 技术方案到交付

架构设计

我们在技术方案中，扩展出执行清单，然后通过codebuddy，基于模版化技术方案 + 模版化执行清单（测试执行清单、发布执行清单）+ 模版化prompt，分阶段实现从研发到部署交付的目标。

- 研发阶段：基于代码层级结构、控制器层、业务层、持久层完成代码的生成及单元测试覆盖及验证

- 测试阶段：基于七彩石配置、DDL/DML工单、蓝盾流水线、lego环境创建等MCP工具实现测试环境的准备，基于自动化测试用例完成代码质量的验证

- 部署阶段：基于七彩石配置、DDL/DML工单等MCP工具完成发布前的准备，基于无人值守，完成智研发布单的申请、执行

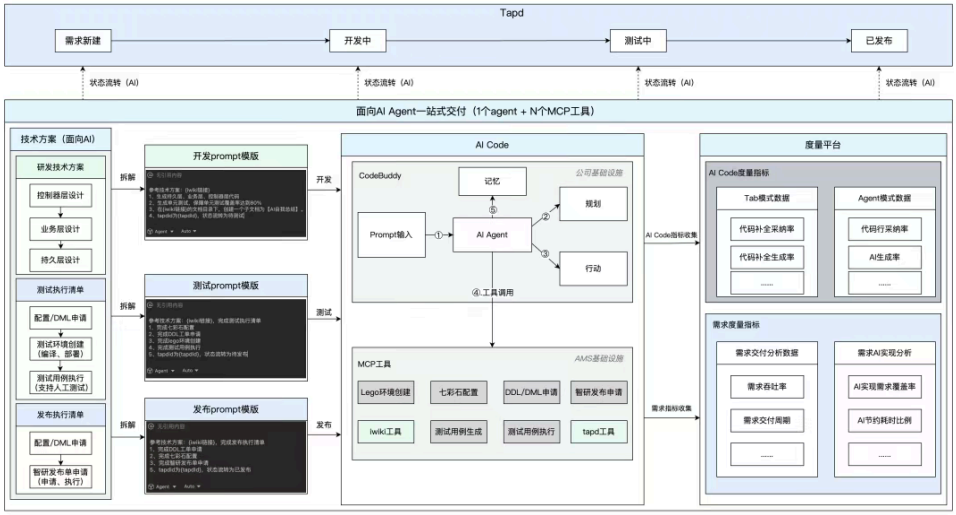


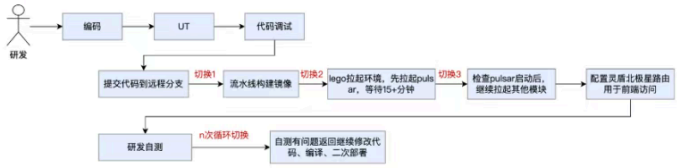
图2.2 审核技术方案到交付的研发流程图

升级事项

我们联通测试、运维团队，通过建设LegoMcp/Skills、DDLMcp/Skills、接口自动化测试Mcp/Skills、接入无人值守，实现研发到部署发布的打通，为后续全自动化交付奠定基础。

Lego环境创建及更新

通过建设Lego环境的DDL、Mcp工具，支持用户在codebuddy内部一键拉起一套测试环境，整体耗时大概在5到10分钟，0平台切换

<table>
<tr><td>之前</td><td>
环境拉起操作路径长，多页面切换，等待时间长（平均耗时25min+， 3*n次切换）


1. 通过100+流水线打包部署 & 缺少统一维护，研发需要感知流水线存在，编译打包时间长
2. LEGO环境模版混乱，20+模版缺少统一维护
3. 研发自测需要在多个页面间来回切换，需要3*n次切换</td></tr>
<tr><td>现在</td><td>
支持codebuddy/anydev环境一键拉起一套环境，平均耗时5~10分钟，0次切换
<table>
<tr><td>环境创建</td><td>服务拉起</td><td>服务更新</td></tr>
<tr><td>codebuddy内部一句话创建测试环境
 
</td><td>  </td><td>  </td></tr>
</table>
</td></tr>
</table>

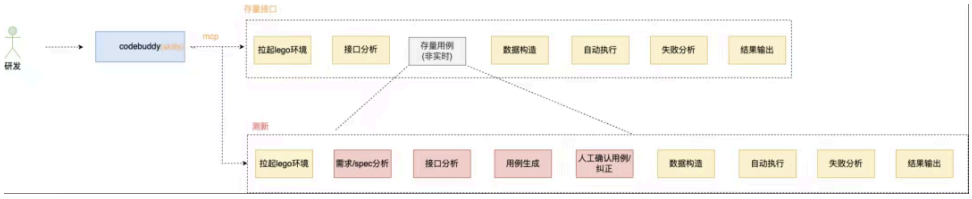
DDL申请

通过建设DDLSkill、DDLMcp工具，将DDL申请耗时降低到秒级以内

<table>
<tr><td>之前</td><td>
DDL申请耗时较长，需要人工填写字段/表格，耗时大概在5到10分钟
<table>
<tr><td>DDL申请</td><td>DDL确认</td><td>DDL执行</td></tr>
<tr><td>1、选择业务域、选择数据库、填表格信息
 
2、 填写字段信息，手动一个个录入
  </td><td>DDL申请前的DDL语句确认
  </td><td>leader审批通过后，执行DDL工单
● 审批列表
 
● 审批详情
</td></tr>
</table>
</td></tr>
<tr><td>现在</td><td>
支持在Codebuddy内部一键申请DDL工单，耗时在秒级
<table>
<tr><td>DDL申请</td><td>DDL确认</td><td>DDL执行</td></tr>
<tr><td>codebuddy内部一句话申请一个ddl工单
 
</td><td>  </td><td>1.审批成功信息
 
2.审批列表
 
3.审批详情
  </td></tr>
</table>
</td></tr>
</table>

接口自动化测试

以审核的申诉复审场景为切入点，通过AI自动识别代码，生成存量接口/增量接口的自动化测试用例接口，最后通过codebuddy mcp一键发起接口自测，实现接口的自动化测试



添加MCP	添加Skills	开始测试
appeal-test-http Tools: query_appeal_elements batch_submit_appeal get_test_summary get_test_report generate_and_run_tests generate_test_data run_test_suite	在codebuddy内部安装测试用例执行skills	prompt输入：测试申诉复审接口

注明：在审核申诉复审场景中，我们当前通过“人工梳理结合AI辅助”的方式，完成了全链路自动化测试用例的构建。目前尚未具备由AI端到端自动生成完整测试用例的能力，该方向存在较大的迭代与优化空间。

接入无人值守

<table>
<tr><td rowspan="3">之前</td><td colspan="2">• 发布的时候需要手动编译、填写智研发布单，单个服务耗时大概在30分钟</td></tr>
<tr><td>提交MR</td><td>生成智研发布单</td></tr>
<tr><td>终端执行MR命令：git leflow mr –t "--story=123456 申诉复审聚合提交功能实现"</td><td>1、 编译流水线
2、 创建智研发布单
3、 执行智研发布单</td></tr>
<tr><td rowspan="3">现在</td><td colspan="2">• 在codebuddy内部提交MR，MR后自动触发编译，自动生成智研发布单，单个服务等待耗时降低到秒级</td></tr>
<tr><td>提交MR</td><td>生成智研发布单</td></tr>
<tr><td>无引用内容
执行终端命令：git leflow mr -t "--story=129872840 【模块】知识库会话管理验证"</td><td>1、MR通过后自动编译流水线，生成智研发布单
智研群助手
请尽快确认发布评审相关信息: 无人值守发布v2-审核对外服务-2026-01-12_18
智研群助手
发布评审[无人值守发布 v2-审核对外服务-2026-01-12_18]
确认地址:http://[redacted]/cicd/3223/release/#/release/detail/200647
----------应用[review_http_server_inner_prod]----------
TAPD 需求相关信息待确认人:
jacksondeng
[redacted]
2、 智研发布单审核通过后，自动部署
  </td></tr>
</table>

架构升级收益

○ 跨平台减少：架构优化前，完成测试和部署链路的工作需要跨6个平台，优化后可以在CodeBuddy内部闭环完成测试和部署的所有操作

○ 操作步骤减少：从测试到部署的步骤从12个步骤优化到5个步骤（理想可优化为2个步骤），提升了研发的交付效率

○ 耗时减少：架构优化前，测试阶段环境创建/更新平均节约时长1h，节约比例达到60%+（暂未统计自动化测试部分）；部署阶段平均节约时长达到0.5d，节约比例达到50%

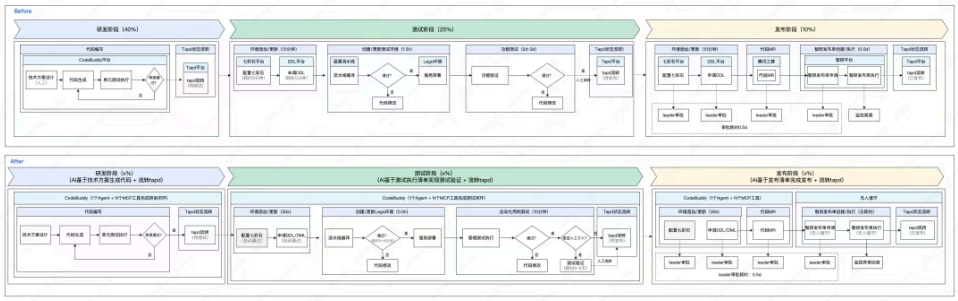


图2.3 技术方案到部署交付流程图

2.1.3需求到交付

架构设计

在实现技术方案至部署交付的链路贯通后，我们将重心转向需求至代码生成链路的建设。最终，通过串联这两大阶段，我们将形成从需求到部署的端到端全链路闭环，从而为构建未来的AI全自动化交付体系奠定坚实基础。具体开发流程如下：

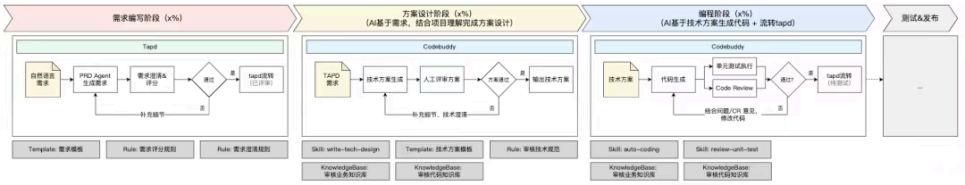
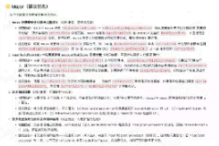


图2.4 需求到代码生成流程图

最佳实践

阶段	说明	产出物
场景	场景1 - 本地模式 场景2- Anydev 模式	/
需求编写&澄清	1、 产品同学使用PRD-Agent，基于需求模板完成需求PRD编写； 2、 基于TAPD的AI评审能力进行需求评审与澄清； 3、 完成需求评审后，在需求中补充相关澄清内容（如业务边界条件、技术细节说明和上下游依赖说明等等）；	标准需求tapd
方案设计	使用技术方案生成skill，同时结合知识库（如审核业务知识库），完成技术方案设计，并输出到iWiki https://[redacted]/tapd_fe/20455692/story/detail/1020455692129 6，基于这个需求完成技术方案设计，存放在https://iwiki.[redacted] com/p/4019518394 已使用 1 条引用 广告审核业务知识库	技术方案 
方案评审	使用技术方案评审skill，对生成的技术方案进行评审，生成评审报告输出到iWiki **门禁逻辑**：方案评审问题会划分成（**Critical / Major / Minor**）三个等级；如果存在Critical等级问题，在OpenClaw模式下会自动打回重新进行设计； 技术方案：：https://[redacted]/p/4019518773 需求：https://[redacted] /tapd_fe/20455692/story/detail/10204556921299[redacted] iWiki 目录：https://[redacted]/p/4019518394? from=recent_view 结合上述信息，完成技术方案评审 已使用 1 条引用 广告审核业务知识库	方案评审报告 

编码	使用代码生成skill，基于前面生成的技术方案，完成编码 技术方案：：https://[redacted]/p/4019518773 需求：https://[redacted] com/tapd_fe/20455692/story/detail/10204556921299(结合上述信息，完成编程	代码diff文件
CodeReview	使用代码CRSkill，对代码进行Review，并输出代码评审报告到iWiki； **门禁逻辑**：CodeReview问题会划分成（**Critical / Major / Minor**）三个等级；如果存在Critical等级问题，在OpenClaw模式下会自动打回编码阶段进行修改； 技术方案：：https://[redacted]/p/4019518773 需求：https://[redacted] com/tapd_fe/20455692/story/detail/1020455692129 ，父目录：https://[redacted]/p/4019518394 在当前分支完成 CodeReview	CR报告（第一轮）

能力沉淀

在需求至代码生成的实践闭环中，我们已构建起一套完整的能力体系，沉淀出1个PRD-Agent、5套规范化标准、3个知识库与3项核心技能。该体系具备良好的通用性与可扩展性，已能覆盖并支持多数工程场景的快速落地。详情如下：

类型	名称	功能描述
agent	PRD-Agent	主要负责审核需求的标准化生成、需求的澄清、需求的打分
规范	需求模板	主要通过需求概述、需求正文、版本管理、外部依赖等4个部分对需求文档进行规范性约束，提升需求的质量
	需求澄清规则	通过对需求概述、需求正文、版本管理、外部依赖等4个部分，建立一套需求澄清的规则，在需求编写不满足条件的情况下，给出修正建议
	需求评分规则	通过对需求背景、价值衡量、解决方案、影响范围、需求正文设定一套标准的打分规则，方便衡量需求的输出质量
	技术方案模板	主要在代码层级结构设计、业务层设计、持久层设计等设定一些规范
	技术规范	通过在基础框架、数据存储、消息队列、监控告警、线上发布等设定了一些技术规范
知识库	审核业务知识库	通过需求Tapd、产品的iwiki文档、研发的iwiki文档，给大模型提供业务知识
	审核代码知识库	通过将审核代码大仓代码提炼出类、方法等摘要信息，给大模型提供代码知识
	审核索引知识库	通过建立本地代码库索引，提升大模型检索代码的效率
技能	技术方案生成skill	从一条需求出发，自动完成从代码理解 → 链路追溯 → 方案编写 → 发布 iWiki 的全流程。方案中每个类名、方法名、表名都有代码搜索证据支撑，不凭空推测
	代码生成skill	用于指导 Coding Agent 按照规范化流程进行代码开发，涵盖需求理解、开发计划制定、代码编写、Proto 桩文件生成以及 Bazel 编译验证的完整开发闭环
	单测生成skill	审核业务单测生成 Skills 是一套标准化的单元测试生成流程规范，专为广审业务代码设计。它整合了单测指引手册、自动化工具和质量检查流程，帮助开发者快速生成高质量的单元测试代码

2.2AI全自动化交付（L3）

架构设计

我们以 CodeBuddy 为基座，完成了从需求到部署交付的实践，但是发现人工干预的环节还是比较多。在此基础上，我们引入了 Harness Engineering for AI Coding 理念，构建了一套标准的工程交付框架。该框架实现了从需求、方案、开发、测试到发布的全流程自动化串联；通过需求评审、方案评审、质量门禁与 MR 等工程卡点保障核心质量；并具备闭环反馈与卡点自动修复能力，支撑流程向自适应、无人值守方向演进。同时，我们持续沉淀标准 Rules、高质量知识库与可复用的 Skills，以此优化 AI 的上下文质量，系统性地增强大模型在复杂交付链路中的决策与执行可靠性。

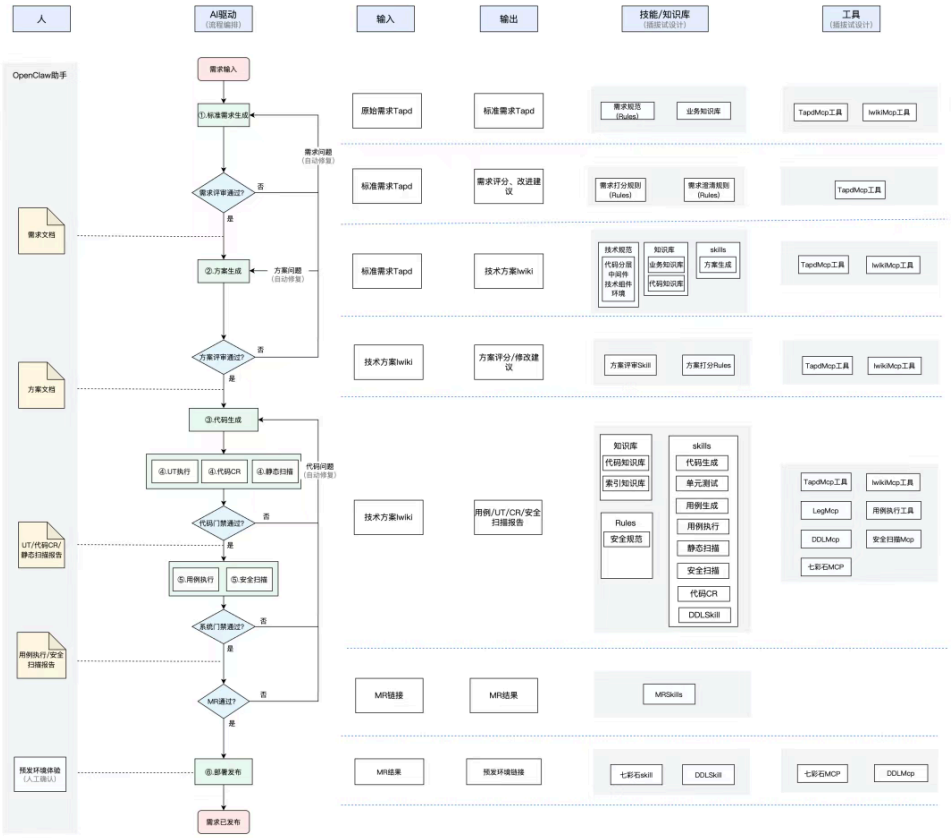


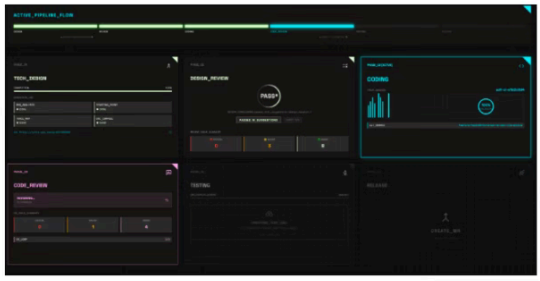
图2.5 AI全自动化架构图

最佳实践

我们通过在审核挑选6个试点需求，基于openclaw完成需求到代码生成的试验，最终沉淀出一套标准的交付框架与能力，方便后续在全中心推广。

阶段	说明	产出物
场景	场景1：本地模式 场景2：Anydev 模式	/
需求编写&澄清	1、 产品同学使用PRD-Agent，基于需求模板完成需求PRD编写； 2、 基于TAPD的AI评审能力进行需求评审与澄清； 3、 完成需求评审后，在需求中补充相关澄清内容（如业务边界条件、技术细节说明和上下游依赖说明等等）；	标准需求tapd
方案设计	使用技术方案生成skill，同时结合知识库（如审核业务知识库），完成技术方案设计，并输出到iWiki 	技术方案 
方案评审	使用技术方案评审skill，对生成的技术方案进行评审，生成评审报告输出到iWiki **门禁逻辑**：方案评审问题会划分成（**Critical / Major / Minor**）三个等级； 如果存在Critical等级问题，在OpenClaw模式下会自动打回重新进行设计； 	方案评审报告 

		
编码	使用代码生成skill，基于前面生成的技术方案，完成编码 	代码diff文件
CodeReview	使用代码CRSkill，对代码进行Review，并输出代码评审报告到iWiki； **门禁逻辑**：CodeReview问题会划分成（**Critical / Major / Minor** ）三个等级； 如果存在Critical等级问题，在OpenClaw模式下会自动打回编码阶段进行修改；  **CodeReview 结论：❌【不通过】** **CR 报告：** **功能审查**：技术方案共 10 项功能要点，9 项一致，1 项不符合（精度处理偏差） **问题摘要：** • 🔴 Critical: **1 项** — PipelineVOBuilder 灰度通道曝光分未做精度处理（ `currentScore / modelRiskFactor` 直接返回 double，未使用 BigDecimal 控制精度，与 post-processor 行为不一致，可能导致前端展示异常或阈值判断浮点误差） • ⚠️ Major: **2 项** — reuse 场景额外 RPC 调用、异常日志误导 • 💡 Minor: **2 项** — import 排序、灰度通道 accountRiskScore 为 null	CR报告（第一轮）

	存在 1 项 Critical 级别问题，请修复 `PipelineVOBuilder.java` 灰度分支的精度处理后重新提交审查。修复方法：在灰度分支的除法结果上增加 `BigDecimal.valueOf(result).setScale(1, RoundingMode.HALF_UP).doubleValue()` 。	
门禁不过（打回编码）		代码diff文件
CodeReview（二轮验收）		CR 报告第二轮
实时进度观测	OpenClaw模式下，提供了进度监控页面，方面查看进展 	--

能力沉淀

通过AI全自动化实践，我们已沉淀出一套标准交付流程、三套核心技能，并实现了与Tapd的深度集成，建立起规范化的需求管理链路。目前，该能力体系已具备较强的工程普适性，可支持多数业务场景的快速接入与应用。详情如下：

类型	名称	功能描述
技能	自动开发流程编排Skill	Auto Dev Flow 是基于 OpenClaw 平台构建的 AI Skill，由 codebuddy-cli 驱动核心阶段执行。它将传统软件开发中从需求分析到代码发布的多个手动环节，编排成一条全自动化流水线，整个流程除首次确认工作区信息外，全程自动推进，并通过消息推送实时同步进度。
	技术方案评审skill	对技术方案进行 8 维度全面评审，输出分 3 级（Critical / Major / Minor）的改进项报告，并自动发布评审意见到 iWiki
	代码CRSkill	深度代码审查工具：分析 Git 变更，评估逻辑、代码规范、安全及需求一致性，支持生成 CodeReview 报告并发布至 iWiki
流程	需求流程自动管控	联同QA、研效同学共建审核标准化的需求管控流程 1、 分支拉取后，tapd自动流转到【开发中】 2、 分支push，tapd自动流转到【测试中】 3、 自动测试用例执行完成，tapd自动流转到【测试完成】 4、 分支MR提交，tapd自动流转到【待发布】 5、 服务发布完成，tapd自动流转到【已发布】
数据	数据指标	1、 AI生成需求的覆盖率（使用PRD-Agent生成的需求数 / 总需求数） 2、 面向AI的需求准备率（采用需求直接生成代码的需求数 / 总需求数）

2.3、效果数据

基于OpenClaw/CodeBuddy模式，我们完成了6个试点需求的三轮迭代优化且成果显著，需求评分稳定在80分以上，技术方案评分达80分以上；平均生成对话轮次2轮；代码行采纳率达90%以上，AI生成率达到80%以上。

3、实践总结

在基于OpenClaw/CodeBuddy模式验证需求到部署的全链路过程中，我们遇到了诸多挑战。目前，大部分问题已得到有效解决，这为我们在中心内的顺利推广奠定了坚实基础。为系统沉淀经验，我们分别从需求侧与编码侧对典型问题进行了梳理与总结，具体如下：

环节	问题类别	具体问题表现	对应解法与实践
需求阶段	信息完备性	1. 外部依赖缺失：接口、库表、服务等信息未明确 2. 异常分支缺失：未描述异常或边界情况处理流程 3. 状态流转缺失：涉及状态机变更时，流转详情不清晰 4. 性能要求缺失：非功能性需求（如性能指标）未定义 5. 业务细节模糊：如金额单位、审批流关键字段等描述不清 6. 可量化性不足：“统计要求”等可量化指标得分普遍偏低	1. 优化需求模板：在模板中强制/建议添加“外部依赖”、“非功能性要求”等字段 2. 增强PRD-Agent评审规则：将异常分支、状态流转、可量化指标等设为必澄清项 3. 联动工具团队：基于 WeDev 研效生态的工具打通能力，与 TAPD、iWiki 团队协作，实现需求与文档的数据互通，解决 MCP 工具无法识别、抓取 iWiki 链接内容的问题
	结构与管理	1. 需求颗粒度不当：产品级需求过大，未拆分，导致技术方案庞大，单人评审困难	1. 探索AI拆分：尝试由AI将大需求拆分为原子化的子需求 2. 调整协作流程：采用“一人生成，多人评审”模式
代码阶段	上下文与知识	1. 代码混淆：对项目内名称相似的文件/类理解错误。 2. 知识检索失效：知识库检索效果差，或引用多知识库时偶发不生效 3. 外部知识缺失：无法获取需求中引用的iWiki接口文档详情。 4. 知识库内容质量问题：已有逻辑需人工确认；复杂流程图AI理解有偏差	1. 开启本地代码索引：为AI提供项目级代码上下文，解决混淆问题。 2. 建设与优化知识库：提升内容质量与RAG检索友好性 3. 开发增强Skill：开发代码生成Skill，优先分析代码再生成方案；开发方案评审Skill，通过门禁机制，在接口参数缺失时自动查阅iWiki补充 4. 工具联动：推动解决知识库引用失效等工具问题
	系统复杂性	1. 长链路与隐式调用：涉及消息队列等异步链路时，AI梳理易出现过早收敛、链路不完整	1. 建设专项Rules：制定“消息驱动”规则，引导AI遇到消息发送代码时继续向下游分析
	规范与约束	1. 技术规范缺失：AI不理解项目特定的技术选型、组件、环境（如Redis/Pulsar/七彩石）及性能规范，导致“自我发挥”或过度设计 2. 方案臃肿：生成方案事无巨细，阅读成本高	1. 建设技术规范：明确技术栈、组件、性能等所有规范 2. 开发技术方案生成Skill：在技术方案skill中引入技术规范，提升技术方案的规范性，在Skill中约束方案结构，非功能模块按需生成 3. 开发CR-Skill：建设技术方案CR-Skill与代码CR-Skill，用于评审与优化
	生成质量	1. 稳定性问题：同一需求多次生成的技术方案质量波动 2. 细节错误：如默认注释掉外部服务调用代码、流程图 Markdown 无法渲染等	1. 流程优化：通过增加对话轮次，明确要求AI执行特定操作（如去掉注释） 2. 规范细化：在评审规则中，添加流程图等元素的生成格式标准

4、未来展望

经过对L2~L3阶段的实践，然后结合业界的harness理念，我们认为L3阶段可归纳为：AI全自动化交付 = LLM + Harness Engineering（工作流程 + 知识库 + skills管理） + Business Engineering（skills + 知识库的沉淀），具体公式组成如下：

交付流程	公式组成	一句话解释	关键点
AI全自动化交付	LLM	通过 大模型协同N个Agent，驱动AI全自动化流程	1、 支持大模型驱动多个Agent协同工作 2、 支持多个Agent间的上下文功能
	Harness Engineering	提供工作流、知识库、skills的管理能力	1、 支持知识库的管理及知识沉淀 2、 支持skills的版本管理
	Business Engineering	沉淀可复用的rules、skills增强大模型的决策与执行力，同时支持线上问题的自动感知与优化	1、 建立一套标准的技术/需求，并且支持持续迭代演进 2、 建设统一的skills开发标准、迭代演进标准 3、 建设标准的知识沉淀框架，保证知识可以高质量的沉淀和迭代演进

我们认为，一个完整的AI全自动化平台应由两套并行、协同的框架构成：AI全自动化交付框架与AI全自动化治理框架 。

AI全自动化交付框架旨在实现从需求、设计、开发、测试到发布的全流程自动化串联，并通过内置的需求评审、方案评审、质量门禁与MR等工程卡点，结合闭环反馈与自修复机制，保障交付流程的稳定、自适应与高质量。

AI全自动化治理框架则与其并行，通过构建覆盖运行时日志、监测指标、Spec知识与中间产物的全域数据洞察系统，实现线上服务的自动巡检与修复、异常的实时感知与优化，以及知识与Skills的自动化回溯与更新，从而系统性地保障架构、性能与可用性。

二者共同构成**“交付驱动效率，治理保障质量”**的双轮驱动体系，确保AI研发全流程在持续高效运转中，始终朝向有序、可持续的方向演进，如图4.1所示：

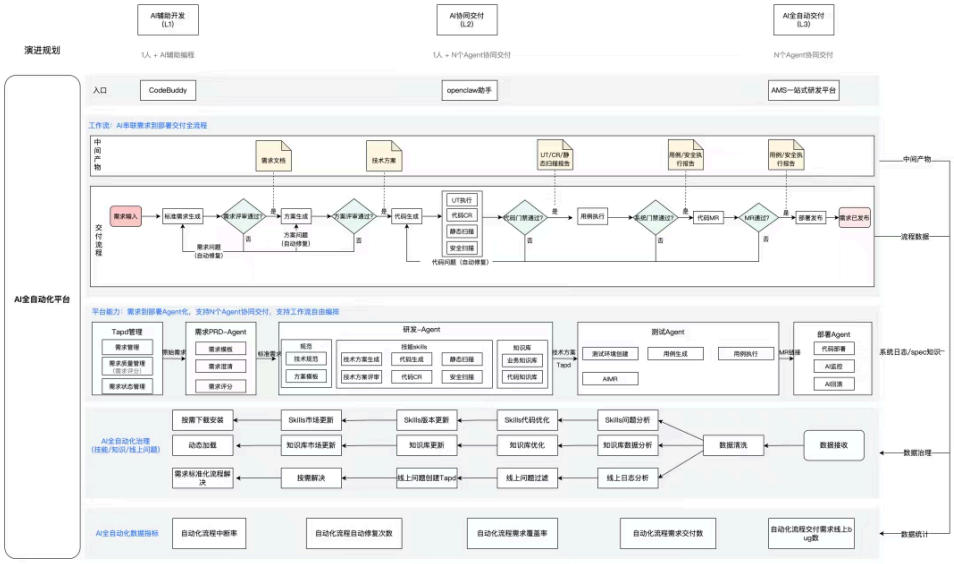


图4.1 AI全自动化平台架构图

5、结语

目前，我们基于OpenClaw/CodeBuddy初步构建了AI全自动化交付框架，但其交付流程与Skills管理仍处于线下、非中心化的初级阶段。同时，AI全自动化治理框架尚在规划中。为实现体系的标准化与规模化，我们计划与研效团队协同，将这两大框架全面整合并迁移至AMS一站式研效平台，以达成线上化、中心化的统一管理。



腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
688篇原创内容
公众号



阅读 1.9万

留言 1

写留言



FUTURE 广东 5月13日 1

赞赞赞！应该还在初版迭代中，缺了：评测、审计、token优化、自我进化等，另外大型系统和大团队协作的场景还有很长的路